

## **1. Εξώφυλλο / Front Page**

**Τμήμα Μηχανικών Πληροφορικής, Υπολογιστών και Τηλεπικοινωνιών**

**Πολυτεχνική Σχολή**

**Διεθνές Πανεπιστήμιο της Ελλάδος**

**Μάθημα:**

**ΜΕΘΟΔΟΛΟΓΙΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ**

**Τελική Αναφορά Εργασίας**

# **Necrosis**

**Συμμετέχοντες:**

**ΠΑΝΤΕΛΕΗΜΩΝ ΖΑΡΑΚΗΣ - 23040**

**ΤΣΑΛΗΣ ΦΩΤΗΣ ΔΗΜΗΤΡΗΣ - 24028**

**ΛΥΜΠΕΡΙΔΗΣ ΒΑΣΙΛΗΣ - 23087**

**Ημερομηνία ολοκλήρωσης:**

**30/05/2026**

## 2. Εισαγωγή

Το Necrosis είναι ένα survival-horror text-based adventure παιχνίδι γραμμένο σε Java. Ο παίκτης ξυπνά μέσα σε μια εγκατάσταση βιοϊατρικής έρευνας, τη Zaun Biomedical, χωρίς να θυμάται τι έχει συμβεί τις προηγούμενες ώρες. Η εγκατάσταση έχει καταρρεύσει, υπάρχουν μολυσμένοι εχθροί στους χώρους και ο παίκτης πρέπει να εξερευνήσει, να λύσει απλά περιβαλλοντικά puzzles και να επιβιώσει μέχρι το τέλος της ιστορίας.

Το project δεν φτιάχτηκε μόνο σαν ένα μεμονωμένο παιχνίδι. Ο βασικός στόχος της εργασίας ήταν να υλοποιηθεί ένας επαναχρησιμοποιήσιμος engine για text-based adventure games. Για αυτό το Necrosis χωρίζει τη γενική λογική του engine από το περιεχόμενο του συγκεκριμένου παιχνιδιού. Τα δωμάτια, τα αντικείμενα, οι εχθροί, τα NPCs, οι διάλογοι, οι εντολές και μεγάλο μέρος της ροής ορίζονται σε JSON αρχεία και φορτώνονται όταν ξεκινά το πρόγραμμα.

Ο χρήστης παίζει δίνοντας εντολές κειμένου, όπως look, take flashlight, go north, use generator ή talk to zaun. Οι εντολές περνάνε από parser, εκτελούνται από αντίστοιχες command classes και αλλάζουν την κατάσταση του παιχνιδιού.

Στο report παρουσιάζονται οι απαιτήσεις της εργασίας, η αρχιτεκτονική της λύσης, ο τρόπος που υλοποιήθηκαν τα βασικά συστήματα, το τελικό αποτέλεσμα που βλέπει ο χρήστης, οδηγίες εγκατάστασης και λειτουργίας, παραδείγματα χρήσης και οι πηγές που χρησιμοποιήθηκαν.

## 3. Περιγραφή απαιτήσεων

Η εκφώνηση της εργασίας ζητούσε την υλοποίηση ενός text-based adventure engine σε Java. Το σημαντικό σημείο ήταν ότι ο engine έπρεπε να είναι επαναχρησιμοποιήσιμος και όχι δεμένος αποκλειστικά με ένα παιχνίδι. Δηλαδή, ένα νέο παιχνίδι θα πρέπει να μπορεί να δημιουργηθεί κυρίως με νέα αρχεία δεδομένων, χωρίς να χρειάζεται αλλαγή στις βασικές κλάσεις του engine.

Οι βασικές απαιτήσεις που προκύπτουν είναι οι εξής:

- Υποστήριξη χώρων ή σκηνών, με περιγραφή, εξόδους, αντικείμενα, NPCs και εχθρούς.
- Υποστήριξη αντικειμένων που μπορούν να εξεταστούν, να μαζευτούν ή να χρησιμοποιηθούν.
- Μοντέλο παίκτη με τρέχον δωμάτιο, inventory και κατάσταση.
- Διαχείριση game state, δηλαδή αποθήκευση της προόδου, των flags, των αντικειμένων και της θέσης του παίκτη.
- Επεκτάσιμο σύστημα εντολών, ώστε κάθε εντολή να είναι ξεχωριστή κλάση και να μπορεί να προστεθεί νέα εντολή χωρίς αλλαγή στον parser.
- Υποστήριξη συνωνύμων και aliases, όπως take, grab και pick up.

- Υποστήριξη εντολών με περισσότερα αντικείμενα και απλές προθέσεις, όπως combine item1 with item2 ή use item on target.
- Ορισμός του περιεχομένου του παιχνιδιού εξωτερικά, σε JSON.
- Αποφυγή hardcoded parsing τύπου if command equals συγκεκριμένο string.
- Δυνατότητα παρουσίασης δεύτερου παιχνιδιού που τρέχει στον ίδιο engine.
- Προαιρετική υποστήριξη NPCs με διάλογο και μεταβολή συμπεριφοράς ανάλογα με την κατάσταση.

Στο Necrosis οι περισσότερες απαιτήσεις καλύπτονται άμεσα από την υπάρχουσα δομή. Το περιεχόμενο του βασικού παιχνιδιού βρίσκεται στο resources/gameData.json, ενώ υπάρχει και δεύτερο μικρό demo στο resources/demoGameData.json. Ο engine φορτώνει τα JSON μέσω της GameLoader και δημιουργεί δυναμικά rooms, exits, items, enemies, NPCs, interactables, encounter phases και command definitions.

Το command system είναι data-oriented. Οι command classes δηλώνονται στο JSON με className και aliases. Η CommandRegistryLoader τις δημιουργεί με reflection και τις περνά στον CommandCutter. Έτσι ο parser δεν χρειάζεται να γνωρίζει εκ των προτέρων συγκεκριμένες εντολές του Necrosis.

Οι απαιτήσεις για rooms, items, player, inventory, movement, game state και βασικό interaction έχουν υλοποιηθεί. Επιπλέον υπάρχουν NPCs, διάλογοι με required και forbidden flags, combat, crafting, save/load, command history, πολλαπλά endings και boss encounter με phases.

Από τον κώδικα φαίνεται ότι η λύση έχει προβλέψει επεκτασιμότητα, κυρίως μέσω configurable commands όπως CombineCommand, UseCommand, EnterCommand και ChooseCommand, καθώς και μέσω game-specific extension classes στο package game.necrosis.

## 4. Σχεδίαση λύσης / Αρχιτεκτονική

Η αρχιτεκτονική του project χωρίζει καθαρά τον reusable engine από το περιεχόμενο και τις ειδικές λειτουργίες του Necrosis. Η βασική ιδέα είναι ότι το engine package περιέχει γενικές έννοιες και μηχανισμούς, ενώ το game.necrosis package περιέχει μόνο ό,τι είναι συγκεκριμένο για αυτό το παιχνίδι.

Το πρόγραμμα ξεκινά από το Main.java. Εκεί δημιουργείται το GameWindow, γίνεται redirect του System.out προς το γραφικό παράθυρο και μετά δημιουργείται ένα GameSession. Αν δεν δοθεί path από τα command line arguments, φορτώνεται το resources/gameData.json. Αν δοθεί path, μπορεί να φορτωθεί άλλο JSON, όπως το resources/demoGameData.json.

Το ui.GameWindow είναι το γραφικό περιβάλλον σε Java Swing. Περιέχει ένα terminal-style παράθυρο με output area, prompt και input field. Δεν περιέχει κανόνες παιχνιδιού,

δωμάτια, εχθρούς ή αντικείμενα. Ο ρόλος του είναι να εμφανίζει το κείμενο και να στέλνει την εντολή που πληκτρολογεί ο χρήστης στο GameSession.

Το engine.loader είναι υπεύθυνο για το φόρτωμα του κόσμου. Η GameLoader διαβάζει το JSON με Gson, ελέγχει βασικά required fields και κατασκευάζει τα αντικείμενα του runtime. Η GameConfig κρατά γενικές ρυθμίσεις όπως title, startRoomId, endingFlags και commands. Η CommandDefinition περιγράφει κάθε command class μαζί με aliases και προαιρετικό config. Η LoadedGameData είναι το τελικό πακέτο δεδομένων που επιστρέφεται στο GameSession.

Το engine.model περιέχει τις βασικές οντότητες του παιχνιδιού. Η Room κρατά περιγραφή, exits, items, enemies, interactables, NPCs και hints. Η Item περιγράφει αντικείμενα και αν είναι portable. Η Player κρατά currentRoom, inventory και κατάσταση τραυματισμού. Η Enemy κρατά τύπο εχθρού και αν έχει ηττηθεί. Τα Npc και DialogueEntry καλύπτουν τους διαλόγους. Τα Interactable και Exit χρησιμοποιούνται για puzzles, χρήση αντικειμένων και μετακινήσεις. Τα EncounterPhase και EnemySpawn χρησιμοποιούνται για το boss encounter.

Το engine.core λειτουργεί σαν κέντρο συντονισμού. Η GameSession φορτώνει δεδομένα, φτιάχνει parser, δημιουργεί CurrentGameState και εκτελεί τις εντολές του παίκτη. Η CurrentGameState κρατά την πραγματική κατάσταση της τρέχουσας παρτίδας: rooms, player, flags, item templates, encounter phases, gameDataPath και command history. Η CommandHistory κρατά τις replayable εντολές που χρησιμοποιούνται και από το save/load.

Το engine.parser αναλαμβάνει την ανάλυση των εντολών. Η CommandCutter χωρίζει την είσοδο σε tokens, ψάχνει το μεγαλύτερο alias που έχει δηλωθεί και περνά τα υπόλοιπα tokens στην κατάλληλη command class. Αυτό είναι σημαντικό επειδή υποστηρίζει multi-word aliases, όπως look at και talk to, χωρίς να υπάρχουν hardcoded full command strings. Η CommandRegistryLoader φορτώνει δυναμικά τις command classes από τα JSON definitions.

Το engine.commands περιέχει γενικές εντολές όπως GoCommand, LookCommand, TakeCommand, UseCommand, InventoryCommand, TalkCommand, CombineCommand, SaveCommand, LoadCommand, HistoryCommand και QuitCommand. Αυτές οι εντολές δεν είναι δεμένες μόνο με το Necrosis. Για παράδειγμα το demoGameData.json χρησιμοποιεί πολλές από τις ίδιες κλάσεις με εντελώς διαφορετικό μικρό κόσμο.

Το game.necrosis περιέχει τις ειδικές επεκτάσεις του βασικού παιχνιδιού. Οι StabCommand, FlashCommand και StrikeCommand υλοποιούν κανόνες μάχης για συγκεκριμένους τύπους εχθρών. Η NecrosisTalkCommand επεκτείνει την TalkCommand ώστε η συζήτηση με τον Zaun να ξεκινά το boss encounter. Το ZaunEncounterSystem χειρίζεται τις φάσεις του τελικού encounter. Έτσι το engine μένει γενικό και οι κανόνες του Necrosis παραμένουν σε ξεχωριστό package.

Η ροή πληροφορίας είναι η εξής: το Main ανοίγει το UI και δημιουργεί GameSession. Το GameSession καλεί GameLoader, φτιάχνει CurrentGameState και καταχωρεί commands

στον parser. Όταν ο χρήστης πατήσει Enter, το GameWindow στέλνει το κείμενο στο GameSession. Ο parser βρίσκει την command class, η command διαβάζει ή αλλάζει το CurrentGameState και το αποτέλεσμα τυπώνεται στο System.out, το οποίο εμφανίζεται ξανά στο UI.

## 5. Υλοποίηση

### 5.1 Βασική ροή παιχνιδιού

Το Necrosis δεν έχει κλασικό real-time game loop. Είναι command-driven. Κάθε φορά που ο χρήστης γράφει μια εντολή και πατά Enter, εκτελείται ένας κύκλος επεξεργασίας: normalize input, parse, execute command, update state και εμφάνιση αποτελέσματος. Αυτό γίνεται μέσα στη GameSession.executePlayerCommand.

Αν μια εντολή εκτελεστεί και είναι replayable, αποθηκεύεται στο CommandHistory. Μετά την εκτέλεση, η GameSession ελέγχει τα endingFlags που έχουν φορτωθεί από το JSON. Αν κάποιο ending flag γίνει true, εμφανίζεται μήνυμα ολοκλήρωσης.

### 5.2 Player

Ο παίκτης υλοποιείται από την Player. Κρατά το currentRoom, το inventory και μια απλή κατάσταση υγείας. Η κατάσταση τραυματισμού χρησιμοποιείται όταν υπάρχουν ενεργοί εχθροί και ο παίκτης προσπαθεί να κάνει ενέργειες που δεν είναι μάχη. Το ThreatSystem μπορεί να τραυματίσει τον παίκτη και αν αυτός δεν θεραπευτεί άμεσα με bandages, μπορεί να οδηγηθεί σε game over.

### 5.3 Rooms και map

Ο χάρτης του Necrosis φορτώνεται από το resources/gameData.json και αποτελείται από έξι δωμάτια: exam\_room, lobby, research\_room, generator\_room, boss\_room και exit\_room. Κάθε δωμάτιο έχει exits με κατευθύνσεις, καθώς και προαιρετικά items, enemies, NPCs, interactables και hints. Οι μετακινήσεις γίνονται με GoCommand, το οποίο ψάχνει την έξοδο στο τρέχον δωμάτιο και μεταφέρει τον παίκτη στο targetRoomId, εκτός αν η έξοδος είναι locked.

Η βασική διάταξη είναι: Exam Room προς Lobby, Lobby προς Research Room και Generator Room, και μετά άνοιγμα της βόρειας διαδρομής προς Boss Room. Από το Boss Room μπορεί να ανοίξει η διαδρομή προς Exit Room, ανάλογα με τις επιλογές του παίκτη.

### 5.4 Items και inventory

Τα items ορίζονται στο JSON και δημιουργούνται από την GameLoader ως templates. Όταν χρειάζεται νέο item, όπως το improvised blade από crafting ή το keycard από επιλογή στο τέλος, το CurrentGameState.createItemById δημιουργεί αντίγραφο από το template. Το

TakeCommand μεταφέρει portable items από το δωμάτιο στο inventory, ενώ το InventoryCommand εμφανίζει τι κουβαλάει ο παίκτης.

Το UseCommand χειρίζεται τη χρήση αντικειμένων σε interactables. Υποστηρίζει απλή χρήση, όπως use generator, αλλά και μορφή με πρόθεση, όπως use keycard on hatch ή use fuel canister with generator, εφόσον υπάρχει αντίστοιχο interactable και required item. Για τα bandages υπάρχει ειδικό healing config μέσα στο JSON.

## 5.5 Enemies και combat

Οι εχθροί φορτώνονται ως Enemy objects με id, name, description και type. Η μάχη στο Necrosis υλοποιείται κυρίως σε game-specific commands. Η StabCommand αντιμετωπίζει standard infected και brute, η FlashCommand αντιμετωπίζει clicker με flashlight και η StrikeCommand αντιμετωπίζει ripper με scrap metal ή improvised blade.

Το ThreatSystem λειτουργεί σαν γενικός μηχανισμός κινδύνου. Αν σε ένα δωμάτιο υπάρχουν ενεργοί εχθροί και ο παίκτης προσπαθήσει να κινηθεί, να πάρει αντικείμενο, να διαβάσει, να μιλήσει ή να χρησιμοποιήσει κάτι, το σύστημα μπορεί να τον τραυματίσει ή να τον εμποδίσει. Έτσι ο παίκτης πρέπει πρώτα να λύσει την απειλή και μετά να συνεχίσει την εξερεύνηση.

## 5.6 NPCs και διάλογοι

Τα NPCs υπάρχουν στο JSON ως Npc με λίστα DialogueEntry. Κάθε dialogue μπορεί να έχει requiredFlag και forbiddenFlag. Η TalkCommand βρίσκει το NPC στο τρέχον δωμάτιο και εμφανίζει τον πρώτο διάλογο που ταιριάζει με τα flags. Αυτό επιτρέπει στην Emily και στον Zaun να λένε διαφορετικά πράγματα ανάλογα με την πρόοδο.

Η NecrosisTalkCommand επεκτείνει την TalkCommand. Μετά τη συζήτηση, αν το NPC είναι ο Zaun και δεν έχει ξεκινήσει ήδη το encounter, καλεί το ZaunEncounterSystem.startEncounter. Με αυτόν τον τρόπο η ειδική ιστορική στιγμή του Necrosis δεν μπαίνει μέσα στη γενική TalkCommand του engine.

## 5.7 Puzzles και progression

Η πρόοδος του παιχνιδιού βασίζεται κυρίως σε flags. Για παράδειγμα, το generator interactable απαιτεί fuel\_canister και θέτει power\_restored. Το terminal στο research\_room απαιτεί power\_restored και θέτει terminal\_prompt\_active. Μετά ο παίκτης μπορεί να γράψει enter code 7314, οπότε η EnterCommand θέτει terminal\_unlocked και ξεκλειδώνει τη βόρεια έξοδο του lobby.

Το CombineCommand είναι generic και διαβάζει recipes από JSON. Στο Necrosis υπάρχει recipe που συνδυάζει broken\_bottle και scrap\_metal για να δημιουργήσει improvised\_blade. Το ChooseCommand χρησιμοποιείται στο τελικό σημείο της ιστορίας και υποστηρίζει επιλογές kill, spare και join με διαφορετικά αποτελέσματα.

## 5.8 Boss encounter

Το τελικό encounter με τον Dr. Zaun υλοποιείται από το `ZaunEncounterSystem`. Όταν ξεκινήσει, φορτώνει `enemy waves` από τα `encounterPhases` του JSON. Η πρώτη φάση βγάζει `standard infected`, η δεύτερη `clickers` και η τρίτη `brute`. Μετά από κάθε νίκη, οι `combat commands` καλούν `advancelIfNeeded`, ώστε το σύστημα να προχωρήσει στην επόμενη φάση ή να ανοίξει το `final choice`.

## 5.9 Save, load και history

Το `SaveManager` αποθηκεύει στο `saves/savegame.json` τη διαδρομή του `game data` και το `command history`. Στο `load`, το `GameSession` ξαναφορτώνει το JSON και κάνει `replay` τις αποθηκευμένες εντολές για να ανακατασκευάσει την κατάσταση. Αυτό κρατά το `save` πιο κοντά στη λογική του `engine` και δεν δένει το `save` με ένα συγκεκριμένο `snapshot` όλων των αντικειμένων.

## 5.10 UI και input handling

Το UI είναι `Java Swing` και έχει σκοτεινή `terminal` αισθητική. Το `output` εμφανίζεται σε `JTextArea` και η είσοδος γράφεται σε `JTextField`. Το `GameWindow` δεν ξέρει τι σημαίνει `look` ή `use`. Απλώς παραδίδει το `input` στο `GameSession`. Η επεξεργασία γίνεται στον `CommandCutter`, ο οποίος επιλέγει το μεγαλύτερο `alias` και δίνει τα υπόλοιπα `tokens` στην `command class`.

# 6. Το παραγόμενο αποτέλεσμα

Όταν ξεκινά το πρόγραμμα, ανοίγει ένα παράθυρο με τίτλο `NECROSIS`. Ο χρήστης βλέπει το `intro` του `Zaun Biomedical`, μια σύντομη περιγραφή της κατάστασης και προτεινόμενες πρώτες εντολές, όπως `look`, `help` και `read notebook`. Δεν υπάρχει ξεχωριστό `main menu`. Η νέα παρτίδα ξεκινά αμέσως από το `exam_room`.

Ο παίκτης παίζει γράφοντας εντολές κειμένου στο κάτω μέρος του παραθύρου. Το παιχνίδι απαντά με περιγραφές χώρων, λίστες αντικειμένων, `hints`, διαλόγους και αποτελέσματα ενεργειών. Το βασικό `gameplay flow` είναι εξερεύνηση, συλλογή αντικειμένων, αντιμετώπιση εχθρών, λύση `puzzles` και λήψη επιλογών στο τέλος.

Στην αρχή ο παίκτης βρίσκεται στο `Exam Room`. Εκεί μπορεί να διαβάσει το `notebook`, να πάρει `flashlight`, `bandages` και `broken bottle` και να μιλήσει στην `Emily`. Μετά πηγαίνει στο `Lobby`, όπου υπάρχει `standard infected`. Αν αγνοήσει την απειλή και προσπαθήσει να κάνει άλλες ενέργειες, το `ThreatSystem` τον τιμωρεί. Η σωστή πρόοδος απαιτεί να χρησιμοποιήσει κατάλληλη `combat command`.

Στη συνέχεια ο παίκτης επισκέπτεται `Research Room` και `Generator Room`. Στο `Research Room` υπάρχουν `fuel canister`, `research notes`, `scrap metal`, `terminal`, `clicker` και `Emily`. Στο

Generator Room υπάρχει generator και ripper. Ο παίκτης πρέπει να βρει fuel, να καθαρίσει τις απειλές, να χρησιμοποιήσει το generator, να ενεργοποιήσει το terminal και να εισαγάγει τον κωδικό 7314 για να ανοίξει το north wing.

Το τέλος εξελίσσεται στο Boss Room. Ο παίκτης μιλά στον Zaun, ξεκινά encounter με τρεις φάσεις εχθρών και μετά καλείται να επιλέξει. Η επιλογή spare οδηγεί σε ending\_cure, η επιλογή join οδηγεί σε ending\_evolution, ενώ η επιλογή kill δίνει keycard και ανοίγει τη διαδρομή προς Exit Room. Εκεί το use keycard ολοκληρώνει το survival ending.

Ο παίκτης θεωρείται ότι προχωρά όταν ενεργοποιεί σημαντικά flags όπως power\_restored, terminal\_unlocked, zaun\_defeated και final\_choice\_available. Το παιχνίδι θεωρείται ολοκληρωμένο όταν ενεργοποιηθεί ένα από τα ending flags που ορίζονται στο JSON: ending\_survival, ending\_cure ή ending\_evolution.

## 7. Συμπεράσματα και μελλοντικές βελτιώσεις

Μέσα από την εργασία μάθαμε στην πράξη πώς μπορεί να οργανωθεί ένα project με αντικειμενοστραφή τρόπο και με καθαρό διαχωρισμό ευθυνών. Το πιο σημαντικό κομμάτι δεν ήταν απλώς να υπάρχει ένα playable παιχνίδι, αλλά να μην είναι όλη η λογική γραμμένη πρόχειρα σε μία κλάση. Ο διαχωρισμός σε loader, model, parser, commands, state και game-specific extensions έκανε το project πιο εύκολο να επεκταθεί και να εξηγηθεί.

Μία βασική δυσκολία ήταν να μη γίνει το περιεχόμενο hardcoded μέσα στον engine. Για αυτό τα rooms, items, exits, NPCs, dialogues, interactables, enemy phases και command aliases μπήκαν σε JSON. Άλλη δυσκολία ήταν το command system, επειδή έπρεπε να υποστηρίζει συνώνυμα και εντολές με περισσότερες λέξεις χωρίς να γίνει ένας μεγάλος if/switch parser. Η λύση ήταν το CommandCutter με longest alias match και το CommandRegistryLoader που φορτώνει commands από className.

Δύσκολο σημείο ήταν και η διαχείριση της προόδου. Το παιχνίδι έχει power, terminal, boss phases και endings, οπότε χρησιμοποιήθηκαν flags μέσα στο CurrentGameState. Αυτό κάνει τη ροή πιο ευέλικτη, αλλά χρειάζεται προσοχή ώστε τα flags να έχουν καθαρά ονόματα και να μην μπερδεύονται μεταξύ τους.

Στο μέλλον θα μπορούσαν να γίνουν αρκετές βελτιώσεις. Θα ήταν χρήσιμο να υπάρχουν αυτοματοποιημένα tests για βασικές ροές, όπως φόρτωμα JSON, movement, combat, save/load και demo game. Επίσης θα μπορούσε να προστεθεί επίσημο UML διάγραμμα, καλύτερο error reporting στο JSON validation, περισσότερα demo games και πιο πλούσιο UI με map view ή command suggestions.

Μία ακόμα πιθανή βελτίωση είναι να μεταφερθεί ακόμα μεγαλύτερο μέρος των κανόνων μάχης σε JSON configuration. Τώρα οι ειδικές εντολές μάχης είναι σωστά έξω από το engine, στο game.necrosis package, αλλά εξακολουθούν να έχουν Necrosis-specific κανόνες



στον κώδικα. Για ένα μεγαλύτερο engine, θα άξιζε να γίνει πιο data-driven και αυτό το κομμάτι.

## 8. Παραρτήματα

### 8.1 Project Plan

| Requirement / Task                                | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8 | Week 9 |
|---------------------------------------------------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| Ανάλυση απαιτήσεων                                | •      | •      | •      | •      | •      | •      | •      | •      | •      |
| Σχεδιασμός αρχιτεκτονικής engine                  | •      | •      | •      | •      | •      | •      | •      | •      | •      |
| Καταγραφή λειτουργικών μονάδων                    | •      | •      | •      | •      | •      | •      | •      | •      | •      |
| Project planning & κατανομή ρόλων                 | •      | •      | •      | •      | •      | •      | •      | •      | •      |
| Υλοποίηση βασικών μοντέλων (Room / Item / Player) |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Υλοποίηση Engine Core / Game State                |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Σχεδιασμός Command System                         |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Main loop / command center                        |        |        |        |        | •      | •      | •      | •      | •      |
| Υλοποίηση movement & look                         |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Inventory / take / inspect                        |        |        |        | •      | •      | •      | •      | •      | •      |
| Υλοποίηση Command Parser                          |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Multi-object parsing                              |        |        |        | •      | •      | •      | •      | •      | •      |
| JSON Loader                                       |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Σχεδιασμός δωματίων & αντικειμένων                |        | •      | •      | •      | •      | •      | •      | •      | •      |
| Game rules / behavior                             |        |        |        |        | •      | •      | •      | •      | •      |
| Interaction system                                |        |        |        |        |        | •      | •      | •      | •      |
| NPCs                                              |        |        |        |        |        | •      | •      | •      | •      |
| Synonyms                                          |        |        |        |        |        |        | •      | •      | •      |
| Πλήρες playable game                              |        |        |        |        |        |        |        | •      | •      |
| Change scenario                                   |        |        |        |        |        |        |        |        | •      |
|                                                   |        |        |        |        |        |        |        |        | •      |

### 8.2 Member Work

Ο παρακάτω καταμερισμός παρουσιάζει τη λογική οργάνωση της δουλειάς της ομάδας στο τελικό project.

#### **ΠΑΝΤΕΛΕΗΜΩΝ ΖΑΡΑΚΗΣ - 23040**

- Οργάνωση της αρχιτεκτονικής του engine και διαχωρισμός engine, game content και game-specific extensions.
- Υλοποίηση βασικών game systems, όπως loader, game state, command flow, parser integration, inventory, movement και save/load.
- Σύνδεση των λειτουργιών του Necrosis με το JSON περιεχόμενο και τελικός έλεγχος ότι το παιχνίδι τρέχει end-to-end.
- Συντονισμός τελικής μορφής του project και έλεγχος συμμόρφωσης με τις βασικές απαιτήσεις της εργασίας.

#### **ΛΥΜΠΕΡΙΔΗΣ ΒΑΣΙΛΗΣ**

- Συμμετοχή στο UI και στην οπτική παρουσίαση του παιχνιδιού.
- Υποστήριξη στη terminal-style εμφάνιση, στη ροή εισόδου/εξόδου και στη συνολική εμπειρία χρήστη.
- Έλεγχος αναγνωσιμότητας των μηνυμάτων, της παρουσίασης των εντολών και της λειτουργικότητας του interface.
- Βοήθεια στη διαμόρφωση της τελικής εικόνας του παιχνιδιού ώστε να είναι πιο καθαρή για παρουσίαση.

#### **ΤΣΑΛΗΣ ΦΩΤΗΣ ΔΗΜΗΤΡΗΣ - 24028**

- Υποστήριξη σε επιμέρους λειτουργίες και έλεγχο της ροής του παιχνιδιού.
- Συμμετοχή στο περιεχόμενο του παιχνιδιού, όπως περιγραφές, παραδείγματα εντολών, gameplay checks και τελικό testing.
- Βοήθεια στον εντοπισμό προβλημάτων λειτουργικότητας και στη διόρθωση λεπτομερειών στη χρήση commands.
- Υποστήριξη στην τεκμηρίωση, στις οδηγίες λειτουργίας και στην προετοιμασία της παρουσίασης.

## **9. Οδηγίες εγκατάστασης**

Το project είναι απλό Java/IntelliJ project. Η δομή του module βρίσκεται στον φάκελο necrosis, με source folder το src και external jar το libs/gson-2.10.1.jar.

Απαιτούνται:

- JDK 17 ή νεότερο. Ενώ το IntelliJ project στο .idea/misc.xml είναι ρυθμισμένο με project-jdk-name jbr-25. Στο περιβάλλον ελέγχου έγινε compile με Java 21 javac.
- IntelliJ IDEA ή αντίστοιχο Java compiler.
- Το Gson jar που υπάρχει ήδη στο project: libs/gson-2.10.1.jar.

Άνοιγμα στο IntelliJ:

- Ανοίγουμε ως project τον φάκελο necrosis.
- Ελέγχουμε ότι το src είναι source folder.
- Ελέγχουμε ότι το libs/gson-2.10.1.jar υπάρχει ως library στο module. Στο υπάρχον IntelliJ config υπάρχει ήδη library με όνομα gson-2.10.1.
- Ορίζουμε JDK 17+ στο Project Structure αν δεν έχει οριστεί ήδη.
- Τρέχουμε την main class Main.

Compile από terminal, μέσα από τον φάκελο necrosis:

```
javac -cp libs/gson-2.10.1.jar -d out $(find src -name "*.java")
```

Εκτέλεση του βασικού Necrosis game σε Linux/macOS:

```
java -cp out:libs/gson-2.10.1.jar Main
```

Εκτέλεση του δεύτερου demo game:

```
java -cp out:libs/gson-2.10.1.jar Main resources/demoGameData.json
```

Σε Windows, το classpath separator είναι semicolon αντί για colon:

```
java -cp out;libs/gson-2.10.1.jar Main
```

## 10. Βιβλιοθήκες που χρησιμοποιήθηκαν

Το project χρησιμοποιεί την Java SE πλατφόρμα και την Swing βιβλιοθήκη που ανήκει στο standard Java desktop API. Το UI βασίζεται σε javax.swing και java.awt classes, όπως JFrame, JTextArea, JTextField, JLabel, JPanel, JScrollPane, BorderLayout, Color και Font.

Η μόνη εξωτερική βιβλιοθήκη που υπάρχει είναι η Gson 2.10.1, αποθηκευμένη τοπικά στο necrosis/libs/gson-2.10.1.jar. Χρησιμοποιείται κυρίως από την GameLoader και τις configurable commands για ανάγνωση JSON και μετατροπή JSON objects σε Java objects.

Το save/load γίνεται με Java και Gson, ενώ το UI γίνεται με standard Swing.

- Java / JDK: επιβεβαιωμένο ως Java 17+, IntelliJ project με jbr-25, compile check με Java 21.
- Standard Java APIs: java.io, java.nio.file, java.util, javax.swing, java.awt.
- External library: Gson 2.10.1.
- Build system: απλό IntelliJ/Java project, χωρίς Maven και χωρίς Gradle.

## 11. Οδηγίες λειτουργίας

Μετά την εγκατάσταση ή το compile, ο χρήστης τρέχει την Main class. Το παιχνίδι ξεκινά αυτόματα νέο game από το startRoomId που υπάρχει στο JSON. Για το βασικό παιχνίδι αυτό είναι το exam\_room. Το save file δεν φορτώνεται αυτόματα. Αν ο χρήστης θέλει να φορτώσει προηγούμενη πρόοδο, γράφει load μέσα στο παιχνίδι.

Η αλληλεπίδραση γίνεται αποκλειστικά με κείμενο. Ο χρήστης γράφει εντολή στο input field και πατά Enter. Οι βασικές εντολές που υποστηρίζονται στο Necrosis είναι:

- look ή view: εμφανίζει την περιγραφή του τρέχοντος δωματίου, αντικείμενα, NPCs, εχθρούς, εξόδους και hints.
- look at <thing>, inspect <thing>, examine <thing>: εξετάζει αντικείμενο, NPC, enemy ή interactable.
- take <item>, grab <item>, pick up <item>: παίρνει portable αντικείμενο από το δωμάτιο.
- inventory ή inv: εμφανίζει το inventory.
- read <item>: διαβάζει notes ή notebook.
- use <thing> ή use <item> on/with <target>: χρησιμοποιεί αντικείμενα και interactables.
- go <direction>, move <direction>, run <direction>: μετακίνηση σε έξοδο όπως north, south, east ή west.
- talk to <person>: διάλογος με NPC.
- combine <item1> with <item2>: crafting, όπως broken bottle με scrap metal.
- flash <enemy>, strike <enemy>, stab <enemy>: combat commands του Necrosis.
- enter code <code>: εισαγωγή κωδικού στο terminal όταν είναι ενεργό.
- choose <option>: τελική επιλογή όταν εμφανιστεί το final choice.
- save, load, history, quit: αποθήκευση, φόρτωση, ιστορικό εντολών και έξοδος.

Για να προχωρήσει ο παίκτης πρέπει να εξερευνήσει τους χώρους, να διαβάσει στοιχεία, να μαζέψει χρήσιμα αντικείμενα, να αντιμετωπίσει τους εχθρούς με τη σωστή εντολή και να ενεργοποιήσει τα βασικά progression flags. Μια τυπική πορεία είναι: συλλογή supplies στο Exam Room, αντιμετώπιση infected στο Lobby, συλλογή fuel και scrap metal στο Research Room, χρήση generator, άνοιγμα terminal, είσοδος στο Boss Room και τελική επιλογή.

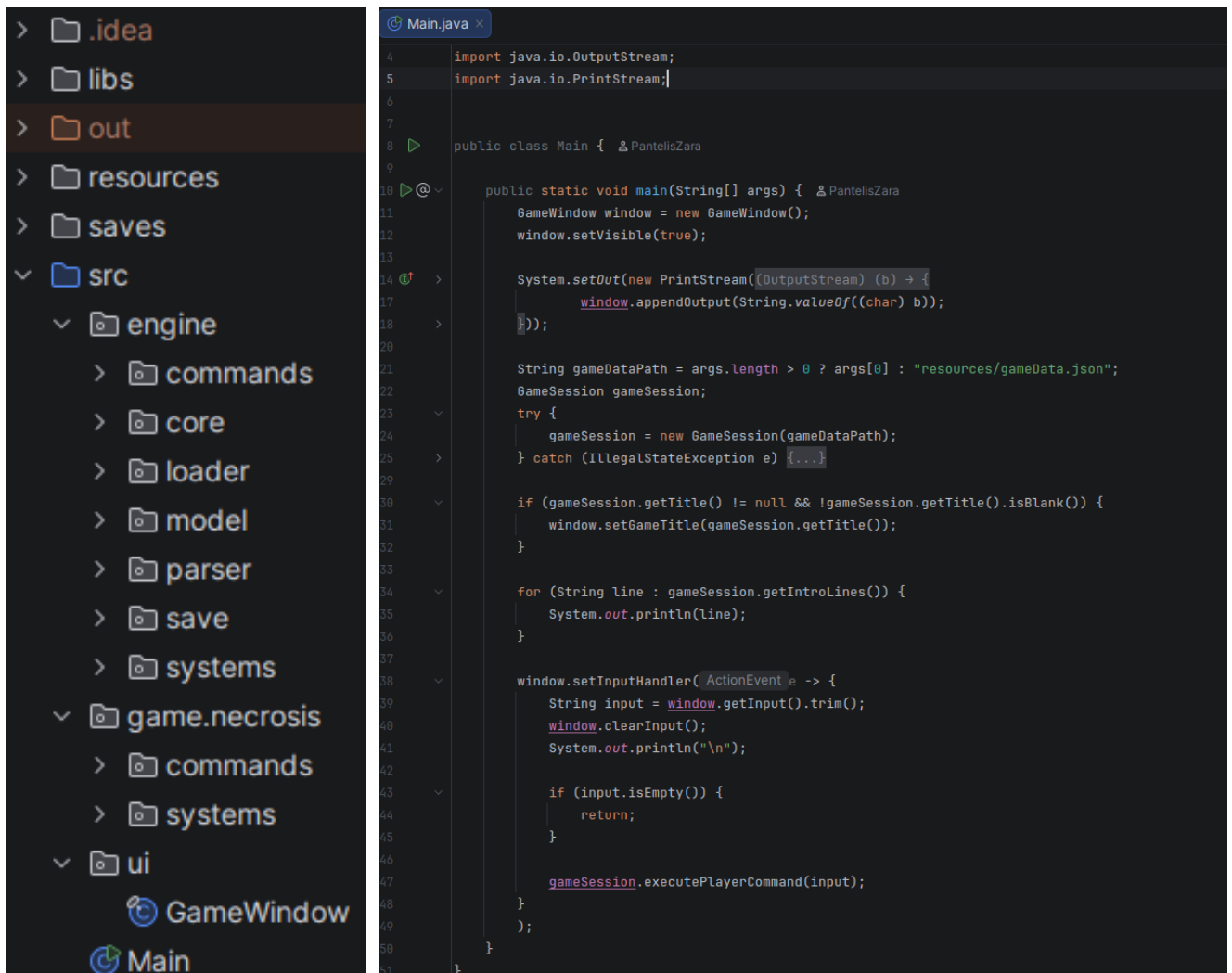
## 12. Ο κώδικας / Παραδοτέα

Το τελικό project περιλαμβάνει τον source code, τα JSON resources, τη βιβλιοθήκη Gson, βοηθητικά αρχεία τεκμηρίωσης και το παρόν τελικό report. Η βασική δομή των παραδοτέων είναι:

- necrosis/src/Main.java: σημείο εκκίνησης του προγράμματος.
- necrosis/src/engine: reusable engine με commands, core, loader, model, parser, save και systems.
- necrosis/src/game/necrosis: ειδικές εντολές και σύστημα encounter για το Necrosis.
- necrosis/src/ui/GameWindow.java: Swing γραφικό περιβάλλον.
- necrosis/resources/gameData.json: κύριο παιχνίδι Necrosis.
- necrosis/resources/demoGameData.json: δεύτερο μικρό παιχνίδι/demo στον ίδιο engine.
- necrosis/libs/gson-2.10.1.jar: εξωτερική βιβλιοθήκη για JSON.
- project\_description.pdf: επίσημη εκφώνηση της εργασίας.

- Necrosis\_Final\_Report.docx: το τελικό Word report.

Υπάρχουν επίσης φάκελοι όπως out και saves, οι οποίοι είναι runtime/build artifacts. Το out δημιουργείται από compile ή IntelliJ και το saves περιέχει αποθηκευμένο παιχνίδι. Δεν αποτελούν τον βασικό source code, αλλά σχετίζονται με την εκτέλεση.



## 13. Τα παραδείγματα

### 13.1 Παράδειγμα εκκίνησης

Μετά την εκτέλεση της Main class, εμφανίζεται το intro του Necrosis. Ο παίκτης μπορεί να ξεκινήσει με:

- look
- read notebook
- talk to emily

Με το look εμφανίζονται η περιγραφή του Exam Room, τα διαθέσιμα αντικείμενα και η έξοδος προς north.

### 13.2 Παράδειγμα συλλογής αντικειμένων

Στην αρχή ο παίκτης μπορεί να πάρει τα βασικά supplies:

```
take flashlight  
take bandages  
take broken bottle  
inventory
```

Το inventory εμφανίζει τα αντικείμενα που κουβαλάει ο παίκτης και τα οποία θα χρειαστούν αργότερα σε μάχη ή θεραπεία.

### 13.3 Παράδειγμα κίνησης

Από το Exam Room ο παίκτης πηγαίνει στο Lobby:

```
go north
```

Αν υπάρχει κλειδωμένη έξοδος, η GoCommand δεν μεταφέρει τον παίκτη και εμφανίζει ότι η διαδρομή είναι locked.

### 13.4 Παράδειγμα μάχης

Στο Lobby υπάρχει standard infected. Αν ο παίκτης έχει broken bottle ή improvised blade, μπορεί να γράψει:

```
stab standard infected
```

Στο Research Room ο clicker αντιμετωπίζεται με flashlight:

```
flash clicker
```

Στο Generator Room ο ripper αντιμετωπίζεται με δυνατό χτύπημα:

```
strike ripper
```

Στο τελικό encounter, ο brute χρειάζεται improvised blade:

```
stab brute
```

### 13.5 Παράδειγμα crafting και inventory

Αφού ο παίκτης βρει scrap metal, μπορεί να φτιάξει improvised blade:

```
take scrap metal  
combine broken bottle with scrap metal  
inventory
```

Το CombineCommand διαβάζει το recipe από το JSON και δημιουργεί το αποτέλεσμα από item template.

### 13.6 Παράδειγμα puzzle προόδου

Μια βασική ακολουθία προόδου είναι η ενεργοποίηση του generator και του terminal:

```
take fuel canister  
go west  
go west  
use generator  
go east  
go east  
use terminal  
enter code 7314
```

Η ακριβής θέση του παίκτη επηρεάζει τις κινήσεις, αλλά η λογική είναι ότι το fuel χρησιμοποιείται στο generator, μετά το terminal ενεργοποιείται και ο σωστός κωδικός ξεκλειδώνει τη βόρεια έξοδο του Lobby.

### 13.7 Παράδειγμα τελικής επιλογής

Μετά τις φάσεις του Zaun encounter, εμφανίζονται διαθέσιμες επιλογές:

```
choose kill  
choose spare  
choose join
```

Το spare εμφανίζει το ending της θεραπείας, το join το ending της εξέλιξης, ενώ το kill δίνει keycard και ανοίγει τον δρόμο προς την έξοδο. Για το survival ending ο παίκτης συνεχίζει προς Exit Room και γράφει:

```
use keycard
```

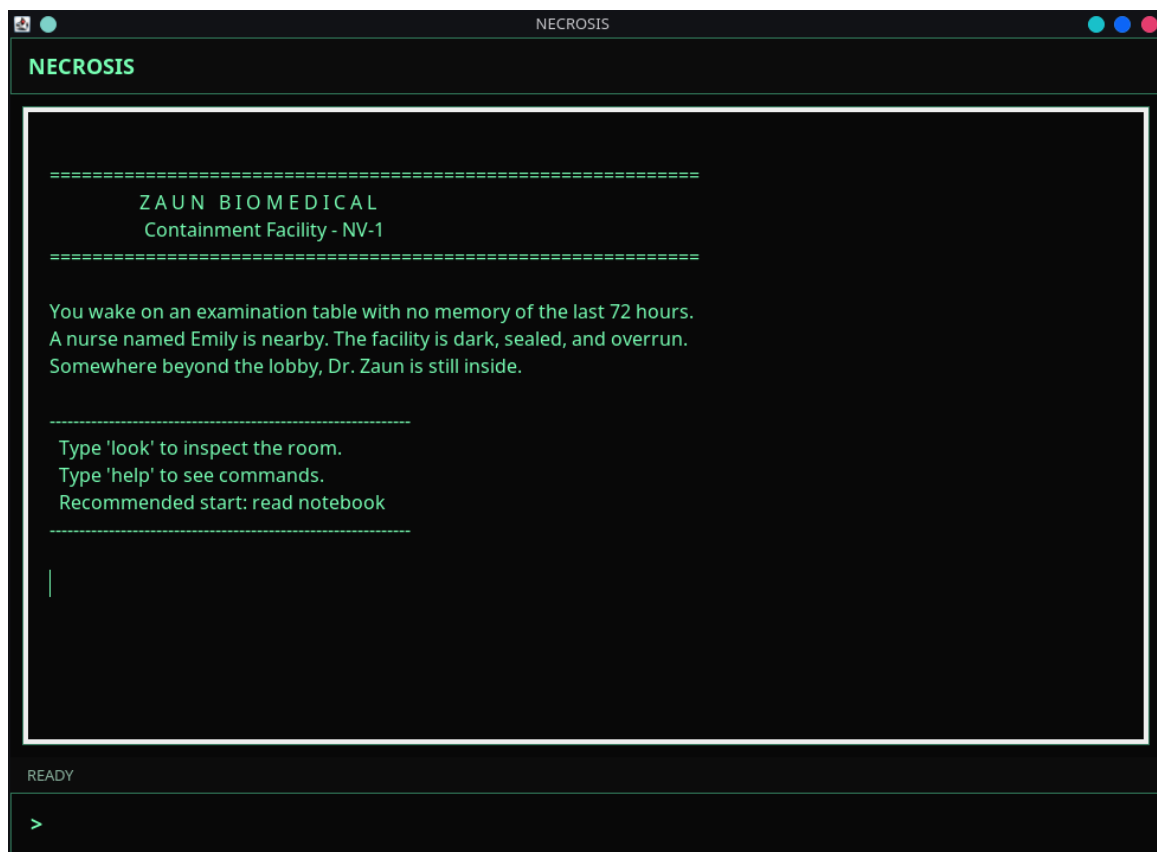
## 13.8 Παράδειγμα δεύτερου demo game

Το demoGameData.json δείχνει ότι ο ίδιος engine μπορεί να τρέξει άλλο παιχνίδι.

Ενδεικτική ροή:

```
java -cp out:libs/gson-2.10.1.jar Main resources/demoGameData.json  
look  
take old note  
go east  
use signal bell
```

## 13.9 Screenshot του παιχνιδιού





## 14.Βιβλιογραφία / Πηγές

- Oracle Java SE 17 API Specification: <https://docs.oracle.com/en/java/javase/17/docs/api/>
- Oracle Swing Tutorial: <https://docs.oracle.com/javase/tutorial/uiswing/>
- JetBrains IntelliJ IDEA Documentation: <https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html>
- Google Gson repository/documentation: <https://github.com/google/gson>
- Η Περιγραφή του project (project\_description.pdf)